Neural Ordinary Differential Equation

MATH797/BDA835

Dr. Xuping Xie

Department of Mathematics & Statistics
Old Dominion University

Neural Ordinary Differential Equations

Neural Ordinary Differential Equations

Neural ODEs

Chen et al., NeurIPS 2018 (Best Paper)

Motivation: From ResNets to Continuous Dynamics

Residual Networks (ResNets) revolutionized deep learning:

$$\boldsymbol{h}_{t+1} = \boldsymbol{h}_t + f_\theta(\boldsymbol{h}_t, t) \tag{1}$$

Key observation: This looks like an **Euler discretization** of an ODE!

ResNet (discrete):

$$\boldsymbol{h}_{t+1} = \boldsymbol{h}_t + f_\theta(\boldsymbol{h}_t, t)$$

- Fixed number of layers
- Discrete time steps
- Each layer has parameters

Euler method for ODE:

$$\boldsymbol{h}(t + \Delta t) = \boldsymbol{h}(t) + \Delta t \cdot f(\boldsymbol{h}(t), t)$$

- Approximates continuous ODE
- Step size $\Delta t = 1$
- What if $\Delta t \to 0$?

Neural ODE idea: Take the continuous limit!
Replace discrete layers with a continuous differential equation.

The Neural ODE Formulation

Definition: A **Neural ODE** defines the hidden state dynamics as:

$$\boxed{\frac{d\boldsymbol{h}(t)}{dt} = f_\theta(\boldsymbol{h}(t), t)} \tag{2}$$

where:

- $\boldsymbol{h}(t) \in \mathbb{R}^d$ is the hidden state at time t
- $f_\theta : \mathbb{R}^d \times \mathbb{R} \to \mathbb{R}^d$ is a neural network with parameters θ
- $t \in [0, T]$ is continuous "time" (depth)

Forward pass: Solve the initial value problem (IVP)

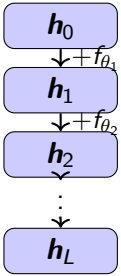
$$\boldsymbol{h}(T) = \boldsymbol{h}(0) + \int_0^T f_\theta(\boldsymbol{h}(t), t)\, dt = \text{ODESolve}(\boldsymbol{h}(0), f_\theta, 0, T)$$

Input and output:

$$\boldsymbol{h}(0) = \boldsymbol{x} \quad \text{(input)} \qquad \boldsymbol{y} = g(\boldsymbol{h}(T)) \quad \text{(output)}$$

Visual Comparison: ResNet vs Neural ODE

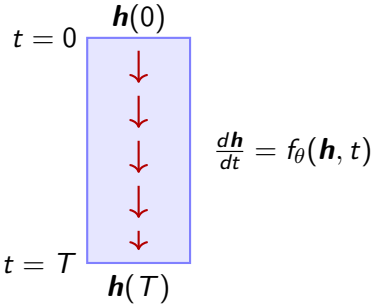
ResNet (Discrete)



L discrete layers

$L \to \infty$

Neural ODE (Continuous)



Continuous dynamics

Key difference: Neural ODE has infinite "layers" with shared parameters θ.

Forward Pass: Numerical ODE Solvers

Problem: We need to compute $\boldsymbol{h}(T) = \boldsymbol{h}(0) + \int_0^T f_\theta(\boldsymbol{h}(t), t)\, dt$

Solution: Use numerical ODE solvers (not just Euler!)

Method	**Update Rule**	**Order**
Euler	$\boldsymbol{h}_{n+1} = \boldsymbol{h}_n + \Delta t \cdot f(\boldsymbol{h}_n, t_n)$	1
Midpoint	$\boldsymbol{h}_{n+1} = \boldsymbol{h}_n + \Delta t \cdot f(\boldsymbol{h}_n + \frac{\Delta t}{2} f_n, t_n + \frac{\Delta t}{2})$	2
RK4	4-stage Runge-Kutta	4
Dopri5	Adaptive step size (default)	5

Adaptive solvers adjust step size based on local error:

- Small steps where dynamics change rapidly
- Large steps where dynamics are smooth
- Accuracy controlled by tolerance parameter

Training

Neural ODE Forward Pass

Given input $\mathbf{x}$, we solve the ODE from $t = 0$ to $t = T$:

$$\frac{d\mathbf{h}(t)}{dt} = f_\theta(\mathbf{h}(t), t), \quad \mathbf{h}(0) = \mathbf{x}$$

Key quantities:

- $\mathbf{h}(t) \in \mathbb{R}^d$ — the **hidden state** at time t (dimension d)
- f_θ — a neural network with parameters θ (e.g., weights, biases)
- $\mathbf{h}(T)$ — the **output** of the Neural ODE
- $L = L(\mathbf{h}(T))$ — the **loss function** (e.g., MSE, cross-entropy)

Training Goal

Compute the gradient $\dfrac{\partial L}{\partial \theta}$ to update θ via gradient descent.

The Challenge: How to Backpropagate?

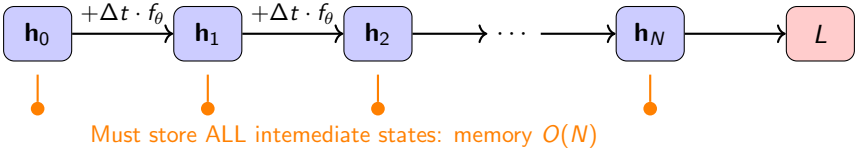
Numerical solvers (Euler, RK4, etc.) compute $\mathbf{h}(T)$ by taking N discrete steps:

$$\mathbf{h}(t_0) \to \mathbf{h}(t_1) \to \mathbf{h}(t_2) \to \cdots \to \mathbf{h}(t_N) = \mathbf{h}(T)$$

Example: Euler method with step size Δt:

$$\mathbf{h}(t_{i+1}) = \mathbf{h}(t_i) + \Delta t \cdot f_\theta(\mathbf{h}(t_i), t_i)$$

This looks like a **residual network** with N layers!



Standard Backpropagation Through the Solver

To compute $\frac{\partial L}{\partial \theta}$, we use the **chain rule** backward through all steps.

The chain rule requires knowing the state at each step:

$$\frac{\partial L}{\partial \theta} = \sum_{i=0}^{N-1} \frac{\partial L}{\partial \mathbf{h}_{i+1}} \cdot \frac{\partial \mathbf{h}_{i+1}}{\partial \theta}$$

where $\frac{\partial \mathbf{h}_{i+1}}{\partial \theta} = \Delta t \cdot \frac{\partial f_\theta(\mathbf{h}_i, t_i)}{\partial \theta}$ $\leftarrow$ needs $\mathbf{h}_i$!

We Must Store All States!

- Computing $\frac{\partial f_\theta}{\partial \theta}$ requires knowing what input $\mathbf{h}_i$ was fed to f_θ
- The gradient depends on **all intermediate activations**
- We must store: $\mathbf{h}_0, \mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_N$

Memory cost: N states $\times$ d dimensions $= \boxed{O(N \cdot d)}$

The Challenge: memory!

Fixed-step solvers (Euler, RK4):

- $N = T/\Delta t$
- Smaller $\Delta t \Rightarrow$ larger N
- More accuracy = more memory

Adaptive solvers (Dormand-Prince, LSODA):

- N varies based on dynamics
- Stiff systems $\Rightarrow$ **huge** N
- N can be 10^3 to 10^6!

Example Memory Calculation

- State dimension: $d = 1000$
- Solver steps: $N = 10000$
- Float32: 4 bytes

$$\begin{aligned}\text{Memory} &= N \times d \times 4 \text{ bytes} \\ &= 10000 \times 1000 \times 4 \\ &= \textbf{40 MB per sample}\end{aligned}$$

Batch of 64: **2.5 GB** just for states!

Problem: Memory scales with solver complexity, not model complexity!

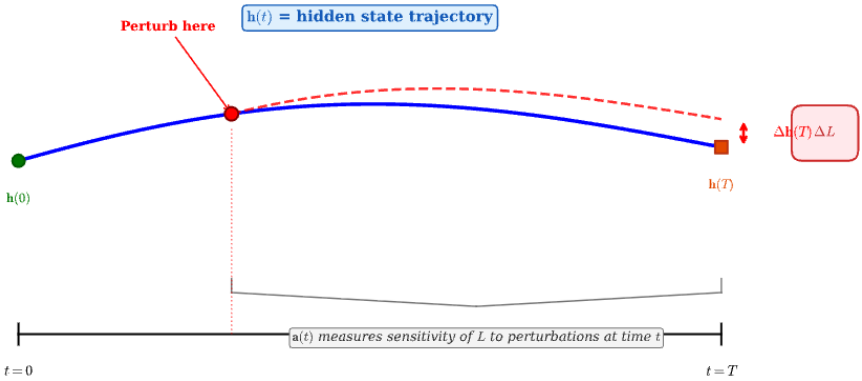
The Adjoint State

The **adjoint state** is defined as the gradient of the loss with respect to the hidden state:

$$\mathbf{a}(t) = \frac{\partial L}{\partial \mathbf{h}(t)} \in \mathbb{R}^d \tag{3}$$

- $L = L(\mathbf{h}(T))$ is the **scalar loss function**
- $\mathbf{h}(t)$ is the **hidden state** (output of the ODE at time t). It's a vector of dimension d
- $\mathbf{a}(t)$ tells us: "How does L change if we perturb $\mathbf{h}(t)$?"

The Adjoint State: Sensitivity



- $\mathbf{a}(t)$ answers: "If I change $\mathbf{h}(t)$ slightly, how much does L change?"
- At $t = T$: $\mathbf{a}(T) = \frac{\partial L}{\partial \mathbf{h}(T)}$ (direct gradient of loss)
- At earlier times: $\mathbf{a}(t)$ accounts for how perturbations **propagate forward**

Deriving the Adjoint ODE

Goal: Find how $\mathbf{a}(t)$ evolves backward in time.

Consider two infinitesimally close times: t and $t+\epsilon$. The hidden states are related by:

$$\mathbf{h}(t+\epsilon) = \mathbf{h}(t) + \epsilon \cdot f_\theta(\mathbf{h}(t), t) + O(\epsilon^2)$$

By the chain rule:

$$\frac{\partial L}{\partial \mathbf{h}(t)} = \frac{\partial L}{\partial \mathbf{h}(t+\epsilon)} \cdot \frac{\partial \mathbf{h}(t+\epsilon)}{\partial \mathbf{h}(t)} \tag{4}$$

$$\mathbf{a}(t) = \mathbf{a}(t+\epsilon)^T \cdot \frac{\partial \mathbf{h}(t+\epsilon)}{\partial \mathbf{h}(t)} \tag{5}$$

Compute the Jacobian:

$$\frac{\partial \mathbf{h}(t+\epsilon)}{\partial \mathbf{h}(t)} = \mathbf{I} + \epsilon \cdot \frac{\partial f_\theta(\mathbf{h}(t), t)}{\partial \mathbf{h}(t)} \tag{6}$$

where $\mathbf{I}$ is the $d \times d$ identity matrix.

Deriving the Adjoint ODE

$$
\begin{aligned}
\mathbf{a}(t)^T &= \mathbf{a}(t+\epsilon)^T \cdot \left(\mathbf{I} + \epsilon \cdot \frac{\partial f_\theta}{\partial \mathbf{h}}\right) \\
&= \mathbf{a}(t+\epsilon)^T + \epsilon \cdot \mathbf{a}(t+\epsilon)^T \cdot \frac{\partial f_\theta}{\partial \mathbf{h}} \\
\mathbf{a}(t)^T - \mathbf{a}(t+\epsilon)^T &= \epsilon \cdot \mathbf{a}(t+\epsilon)^T \cdot \frac{\partial f_\theta}{\partial \mathbf{h}} \qquad (7) \\
-\frac{d\mathbf{a}}{dt} = \lim_{\epsilon \to 0} \frac{\mathbf{a}(t)^T - \mathbf{a}(t+\epsilon)^T}{\epsilon} &= \lim_{\epsilon \to 0} \mathbf{a}(t+\epsilon)^T \cdot \frac{\partial f_\theta}{\partial \mathbf{h}}
\end{aligned}
$$

The Adjoint ODE

$$
\boxed{\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^T \frac{\partial f_\theta(\mathbf{h}(t), t)}{\partial \mathbf{h}}} \qquad (8)
$$

The **negative sign** indicates we solve this ODE **backward in time**!

The Adjoint ODE

$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^T \frac{\partial f_\theta(\mathbf{h}(t), t)}{\partial \mathbf{h}}$$

- $\mathbf{a}(t) \in \mathbb{R}^d$ — adjoint state (what we're solving for)
- $\frac{\partial f_\theta}{\partial \mathbf{h}} \in \mathbb{R}^{d \times d}$ — **Jacobian** of the neural network f_θ. *This is computed via automatic differentiation (PyTorch/JAX).*
- $\mathbf{a}(t)^T \frac{\partial f_\theta}{\partial \mathbf{h}}$ — vector-Jacobian product (VJP). Efficient: $O(d)$, not $O(d^2)$! This is what `backward()` computes.

Terminal Condition

At $t = T$, the adjoint starts from the gradient of the loss:

$$\mathbf{a}(T) = \frac{\partial L}{\partial \mathbf{h}(T)}$$

Advantage

We don't need to **store** intermediate states—we can **recompute** them!

The adjoint method does this:

1. **Forward pass:** Solve ODE from $t = 0$ to $t = T$
   - Only store the final state $\mathbf{h}(T)$ and initial state $\mathbf{h}(0)$
2. **Backward pass:** Solve the **augmented ODE** from $t = T$ to $t = 0$:

$$\frac{d}{dt}\begin{pmatrix}\mathbf{h}\\ \mathbf{a}\\ \frac{\partial L}{\partial \theta}\end{pmatrix} = \begin{pmatrix} f_\theta(\mathbf{h}, t)\\ -\mathbf{a}^T \frac{\partial f_\theta}{\partial \mathbf{h}}\\ -\mathbf{a}^T \frac{\partial f_\theta}{\partial \theta}\end{pmatrix}$$

running **backward in time** (i.e., with negative time step)

No storage of intermediate states needed!

Memory Comparison: Naive vs. Adjoint

Quantity	Naive Backprop	Adjoint Method
Forward states $\mathbf{h}(t_i)$	$O(N \cdot d)$	$O(d)$
Adjoint states $\mathbf{a}(t)$	—	$O(d)$
Parameter gradients	$O(\|\theta\|)$	$O(\|\theta\|)$
Total	$O(N \cdot d + \|\theta\|)$	$O(d + \|\theta\|)$

Naive: $O(N)$ Memory

- Store all N states
- Memory $\propto$ solver steps
- Doesn't scale to deep ODEs

Adjoint: $O(1)$ Memory

- Store only current state
- Memory independent of N
- Scales to any depth!

Trade-off: Adjoint requires **2 ODE solves** instead of 1 (more compute, less memory).

The Complete Adjoint Algorithm

Algorithm 1 Adjoint Sensitivity Method

Forward pass:

Solve $\frac{d\mathbf{h}}{dt} = f_\theta(\mathbf{h}, t)$ from $t = 0$ to $t = T$, $\boldsymbol{h}(T) = \text{ODESolve}(\boldsymbol{h}(0), f_\theta, 0, T)$.

Store only $\mathbf{h}(0)$ and $\mathbf{h}(T)$

Compute loss $L = L(\mathbf{h}(T))$

Backward pass:

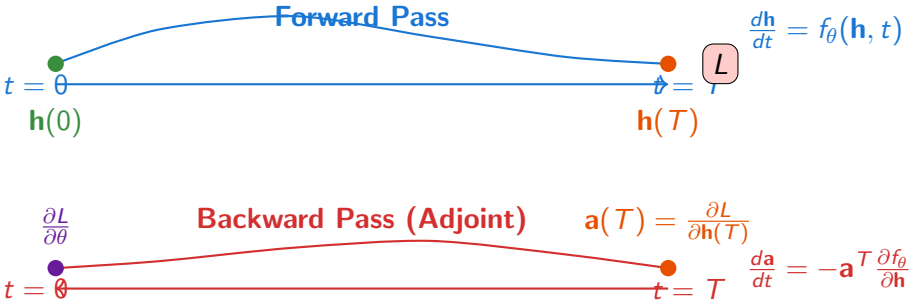
Initialize: $\mathbf{a}(T) = \frac{\partial L}{\partial \mathbf{h}(T)}$, $\quad \frac{\partial L}{\partial \theta}\Big|_{t=T} = \mathbf{0}$

Solve augmented ODE **backward** from $t = T$ to $t = 0$:

$$\frac{d}{dt}\begin{pmatrix} \mathbf{h} \\ \mathbf{a} \\ \frac{\partial L}{\partial \theta} \end{pmatrix} = \begin{pmatrix} f_\theta(\mathbf{h}, t) \\ -\mathbf{a}^T \frac{\partial f_\theta}{\partial \mathbf{h}} \\ -\mathbf{a}^T \frac{\partial f_\theta}{\partial \theta} \end{pmatrix}$$

Output: $\frac{\partial L}{\partial \theta}\Big|_{t=0}$ (accumulated gradient)

Summary



Memory: $O(1)$ — only store current state!

Computational Trade-offs

Property	**ResNet**	**Neural ODE**
Parameters	$O(L)$ (per layer)	$O(1)$ (shared)
Forward memory	$O(L)$	$O(1)$
Backward memory	$O(L)$	$O(1)$ (adjoint)
Compute cost	Fixed	Adaptive
Depth	Fixed L	Continuous $[0, T]$
Invertible	No	Yes

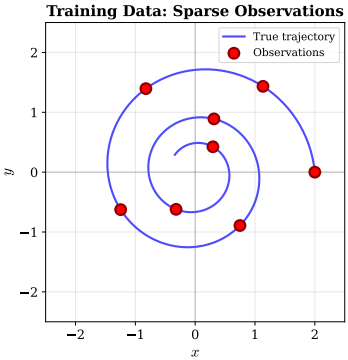
Trade-off:

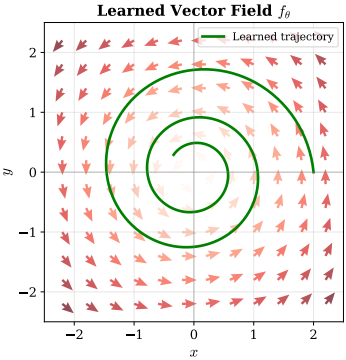
- Neural ODE: Lower memory, but more compute (ODE solver evaluations)
- Best when memory is the bottleneck
- Adaptive solvers use more steps for complex dynamics

Example 1: Learning a Spiral Dynamics

Task: Learn the dynamics of a 2D spiral from data.

True system: $\frac{d}{dt}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} -0.1 & -1 \\ 1 & -0.1 \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix}$ (damped oscillator)

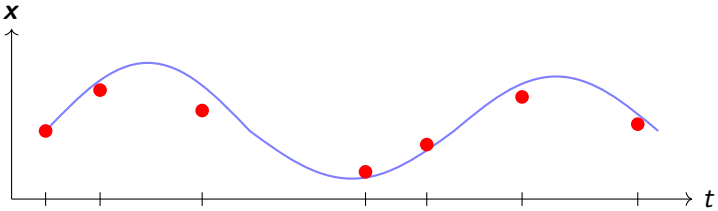




Neural ODE can predict at any time t, not just at observed times!

Example 2: Irregularly-Sampled Time Series

Problem: Real-world data often has irregular time intervals.



Irregular time intervals

Why Neural ODE is natural for this:

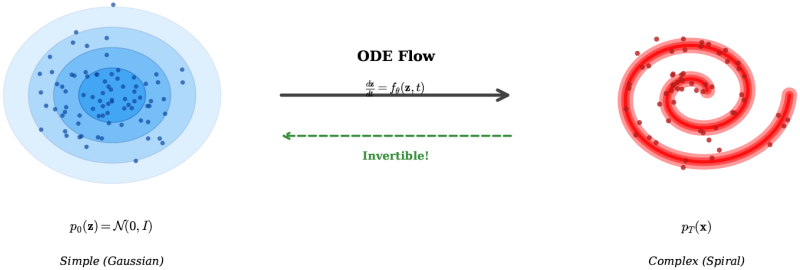
- ODE solvers can evaluate at any time t
- No need to discretize to fixed grid
- Naturally handles missing data

Applications: Medical records, sensor data, financial data, climate data

Example 3: Continuous Normalizing Flows

Goal: Transform a simple distribution (Gaussian) into a complex one.

Key property: Neural ODE flows are **invertible** (diffeomorphisms)!



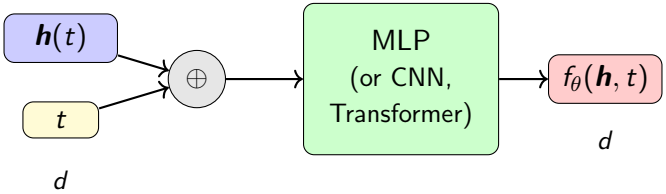
Change of variables formula:

$$\log p_T(\boldsymbol{x}) = \log p_0(\boldsymbol{z}) - \int_0^T \mathrm{Tr}\left(\frac{\partial f_\theta}{\partial \boldsymbol{h}(t)}\right) dt$$

Advantage over discrete flows: No need for special invertible architectures!

Neural ODE Architecture

The function $f_\theta(\boldsymbol{h}, t)$ is a neural network:



Common choices for f_θ:

- **MLP:** $f_\theta(\boldsymbol{h}, t) = W_2\sigma(W_1[\boldsymbol{h}; t] + b_1) + b_2$
- **Time embedding:** Use positional encoding for t
- **Lipschitz networks:** Spectral normalization for stability

Note: Same network f_θ is evaluated at all times (parameter sharing)!

Training a Neural ODE

Training procedure:

1. **Forward pass:** Solve ODE from $t = 0$ to $t = T$

$$\boldsymbol{h}(T) = \text{ODESolve}(\boldsymbol{x}, f_\theta, 0, T)$$

2. **Compute loss:**

$$L = \ell(\boldsymbol{h}(T), \boldsymbol{y}_{\text{target}})$$

3. **Backward pass:** Solve adjoint ODE from $t = T$ to $t = 0$

$$\frac{\partial L}{\partial \theta} = \text{ODESolve}(\text{adjoint system}, T, 0)$$

4. **Update:** $\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$

Implementation (PyTorch):

```
from torchdiffeq import odeint_adjoint as odeint
```

The `odeint` function handles both forward and backward passes automatically!

Neural ODE for Learning Dynamical Systems

The perfect match: Dynamical systems ARE ODEs!

Setup:

- Observe trajectories $\{\boldsymbol{x}(t_0), \boldsymbol{x}(t_1), \ldots, \boldsymbol{x}(t_N)\}$
- Want to learn the underlying dynamics $\frac{d\boldsymbol{x}}{dt} = f(\boldsymbol{x})$

Neural ODE approach:

$$\hat{\boldsymbol{x}}(t_k) = \boldsymbol{x}(t_0) + \int_{t_0}^{t_k} f_\theta(\boldsymbol{x}(t))\, dt$$

Loss:

$$L = \sum_{k=1}^{N} \|\hat{\boldsymbol{x}}(t_k) - \boldsymbol{x}(t_k)\|^2$$

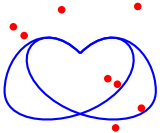
Key advantage: We learn f_θ directly — the dynamics itself!
Can predict at any future time, not just observed times.

Example: Learning the Lorenz System

Lorenz system (chaotic dynamics):

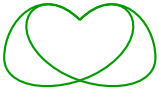
$$\frac{dx}{dt} = \sigma(y - x), \quad \frac{dy}{dt} = x(\rho - z) - y, \quad \frac{dz}{dt} = xy - \beta z$$

Training data:



Noisy observations

Learned attractor:



Neural ODE prediction

Neural ODE learns:

- The vector field $f_\theta \approx (f_1, f_2, f_3)$
- Can extrapolate to unseen time ranges
- Captures the attractor structure

Neural ODE Variants and Extensions

Variant	Description
Augmented NODE	Add extra dimensions: $\boldsymbol{h} \in \mathbb{R}^{d+p}$
Latent ODE	Encode to latent space, evolve with ODE
ODE-RNN	Combine ODE with RNN for updates
Neural CDE	Controlled differential equations (for irregular data)
Neural SDE	Stochastic differential equations
Hamiltonian NODE	Preserve Hamiltonian structure
Lagrangian NODE	Preserve Lagrangian structure
FFJORD	Free-form continuous normalizing flows

Each variant adds inductive biases for specific applications!

Advantages and Limitations

Advantages:

- ✓ Constant memory (adjoint)
- ✓ Continuous-time modeling
- ✓ Invertible by design
- ✓ Handles irregular time series
- ✓ Natural for physics
- ✓ Adaptive computation
- ✓ Preserves topology

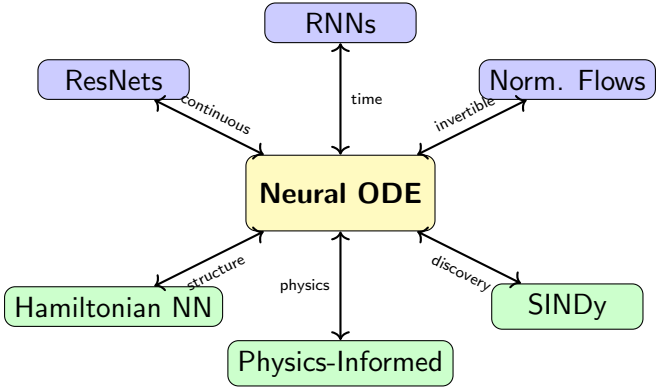
Limitations:

- × Slower than ResNets (ODE solver)
- × Can't represent discontinuous maps
- × Training can be unstable
- × Harder to parallelize
- × Solver tolerance tuning
- × Not always needed

When to use Neural ODEs:

- Modeling continuous dynamical systems
- Irregularly-sampled time series
- Normalizing flows / density estimation
- Physics-informed learning

Neural ODE in the Bigger Picture



Neural ODE bridges:

- Deep learning (ResNets) $\leftrightarrow$ Dynamical systems (ODEs)
- Data-driven $\leftrightarrow$ Physics-based modeling
- Discrete $\leftrightarrow$ Continuous representations

Summary: Neural ODEs

Concept	**Key Point**
Formulation	$\frac{d\boldsymbol{h}}{dt} = f_\theta(\boldsymbol{h}, t)$ — continuous depth
Forward pass	Numerical ODE solver (Dopri5, RK4, etc.)
Backward pass	Adjoint method — $O(1)$ memory
Key property	Flow is a diffeomorphism (invertible, smooth)
Main advantage	Continuous-time, memory-efficient, invertible
Best for	Dynamical systems, time series, normalizing flows

The big idea:

$$\boxed{\text{Neural ODE} = \text{Neural Network} + \text{Differential Equation}}$$

Combines the expressiveness of deep learning with the mathematical structure of dynamical systems!

Key References

Original paper:

- Chen et al., "Neural Ordinary Differential Equations", NeurIPS 2018 (Best Paper)

Extensions:

- Rubanova et al., "Latent ODEs for Irregularly-Sampled Time Series", NeurIPS 2019
- Grathwohl et al., "FFJORD: Free-form Continuous Dynamics", ICLR 2019
- Kidger et al., "Neural Controlled Differential Equations", NeurIPS 2020

Code:

- `torchdiffeq`: `https://github.com/rtqichen/torchdiffeq`
- `diffrax` (JAX): `https://github.com/patrick-kidger/diffrax`